Lezione 7
Contract Testing e Property-Based Testing

Verificare cio che attraversa i confini e cio che deve restare vero per intere classi di input.

Domande centrali
1) Come facciamo a sapere che due componenti che evolvono separatamente sono ancora compatibili?
2) Come verifichiamo una regola quando lo spazio degli input e troppo grande per essere coperto con esempi scelti a mano?

Prerequisito: Lezione 6 - Clean / Hexagonal Architecture: boundaries, ports, adapters, dependency direction e test seams.

Durata: circa 30-45 minuti. **Formato:** teoria, modello mentale, esempi framework-independent, laboratorio e fonti.

Mappa della lezione

1. Perche contract testing e property-based testing arrivano ora
2. Contract Testing: che problema risolve e che cosa verifica
3. Consumer-Driven Contracts, schema contracts e provider verification
4. Property-Based Testing: properties, generators, runner e shrinking
5. Pattern di property: invarianti, round-trip, idempotenza, metamorphic, monotonicita, equivalenza
6. Esempi equivalenti in Python, TypeScript e PHP
7. Come combinare esempi, properties, contracts, integration ed E2E
8. Failure mode e anti-pattern
9. Ruolo dell AI nel workflow Domain -> Requirements -> Spec -> Test -> Agent -> Verify
10. Laboratorio pratico e checklist finale

Idea guida: i test non sono tutti la stessa cosa. Ogni tecnica deve rispondere a una domanda ingegneristica precisa e avere una fonte di verita adeguata.

1. Perche questa lezione arriva dopo Hexagonal Architecture

Nella Lezione 6 abbiamo progettato confini espliciti. Una porta definisce il modo in cui il core applicativo chiede qualcosa al mondo esterno oppure espone una capacita. Un adapter traduce tra il linguaggio del core e un protocollo concreto: HTTP, database, coda, filesystem, SDK esterno, CLI.

Questo crea **test seams**: punti in cui possiamo sostituire una dipendenza, osservare un protocollo o verificare un comportamento senza trascinare l intero sistema. Ma un seam da solo non ci dice *quale tipo di test* scrivere.

Due problemi diventano particolarmente visibili:

- **Compatibilita al boundary:** consumer e provider possono avere entrambi test verdi, eppure non capirsi piu dopo una modifica di path, payload, status code, header o schema evento.
- **Spazio degli input:** una funzione pura puo essere ben isolata, ma pochi esempi scelti a mano non esplorano le combinazioni che fanno emergere edge case.

Contract Testing risponde soprattutto alla prima domanda. Property-Based Testing risponde soprattutto alla seconda. Sono tecniche complementari, non concorrenti.

2. Contract Testing: verificare l accordo osservabile

2.1 Il modello mentale

Un contract test verifica che due parti che comunicano condividano ancora lo stesso accordo osservabile. Le parole consumer e provider sono piu generali di client e server: il consumer e chi dipende da un comportamento; il provider e chi lo offre. La comunicazione puo essere HTTP, RPC, un evento o un messaggio asincrono.

Nell approccio consumer-driven, il consumer esprime le interazioni di cui ha realmente bisogno. L esecuzione dei test del consumer produce un contratto; il provider esegue poi una verifica contro quel contratto. Pact descrive il proprio modello proprio in questi termini: il contratto viene generato dai test del consumer e contiene esempi concreti di request/response o messaggi.

> **Domanda del contract test: “La versione del provider che voglio distribuire continua a soddisfare le aspettative osservabili dei consumer che dichiaro compatibili?”**

2.2 Che cosa e un contratto, e che cosa non lo e

Un contratto puo includere metodo, path, query parameter, header rilevanti, forma minima del body, status code, tipi dei campi, messaggi pubblicati o consumati e alcune regole di matching. Non dovrebbe duplicare tutta la logica business del provider.

Se un consumer usa soltanto **id**, **total** e **currency**, un contract test troppo rigido che pretende anche quindici campi irrilevanti crea coupling artificiale. Una modifica innocua diventa apparentemente breaking.

Questo principio e importante perche un consumer-driven contract non vuole congelare il provider: vuole proteggere le dipendenze reali del consumer lasciando libero il resto.

2.3 Contract test vs integration test vs E2E

Tipo	Scope principale	Domanda	Costo tipico
Unit / example	funzione, classe, use case	la regola locale e corretta?	molto basso
Contract	boundary consumer-provider	siamo compatibili?	basso/medio
Integration	piu componenti reali	collaborano tecnicamente?	medio
End-to-end	sistema completo	il flusso reale funziona?	alto

Una buona suite usa ciascun livello per una domanda diversa. Cercare di dimostrare tutto con E2E produce test lenti e fragili; cercare di dimostrare la compatibilita tra deploy indipendenti solo con unit test lascia un buco reale.

3. Consumer-Driven Contracts e alternative

3.1 Consumer-driven contract (CDC)

Nel CDC il consumer guida il contratto. Un test usa il codice reale del client del consumer contro un mock server controllato dallo strumento. Le interazioni osservate vengono serializzate nel contratto. Successivamente il provider riproduce tali richieste e dimostra di produrre risposte compatibili.

Questo e il caso d uso tipico di Pact. La documentazione Pact sottolinea anche che il test del consumer dovrebbe esercitare il client reale dell applicazione, non un generico HTTP client scritto solo dentro il test: altrimenti si puo produrre un contratto valido senza aver verificato il codice che andra davvero in produzione.

3.2 Schema/API-first contracts

OpenAPI e una specifica language-agnostic per descrivere interfacce HTTP. La versione pubblicata piu recente al momento di questa lezione e OpenAPI 3.2.1, del 10 settembre 2026. Uno schema API-first e utile per documentazione, code generation, governance, validazione e test di conformita.

Ma una descrizione completa dell API e un consumer-driven contract non rispondono alla stessa domanda. Lo schema dice che cosa il provider dichiara di supportare; il CDC puo dire quali parti sono effettivamente necessarie a un consumer concreto.

3.3 Provider-driven contracts e stubs

In altri workflow il provider pubblica stub o contratti, e i consumer si verificano contro tali artefatti. Spring Cloud Contract supporta sia approcci consumer-driven sia provider contract testing con stub. Questo puo essere efficace in organizzazioni coordinate, ma il rischio e che la definizione rifletta soprattutto il punto di vista del provider.

3.4 Contratti di messaggi ed eventi

Il concetto non si limita a HTTP. Un consumer di un evento dipende da tipo del messaggio, campi, semantica e regole di compatibilita. Una modifica di schema evento puo rompere il consumer esattamente come una modifica di risposta HTTP. Pact supporta interazioni HTTP e, nelle specifiche moderne, anche scenari di messaging.

4. Esempio di contract test: Checkout e Pricing Service

Immaginiamo due componenti deployabili indipendentemente. **Checkout** e il consumer. **Pricing Service** e il provider. Il checkout usa una porta applicativa chiamata **QuoteProvider**. L adapter HTTP del consumer traduce una chiamata alla porta in una richiesta POST /quotes.

```
POST /quotes

{
"items": [
{"sku": "A1", "qty": 2}
]
}

HTTP 200
{
"total": 1990,
"currency": "EUR"
}
```

Un contract test sensato protegge almeno: metodo e path; campi della request realmente inviati; status code atteso; campi e tipi della response realmente consumati. Se il provider rinomina **total** in **amount** senza una strategia compatibile, la provider verification deve fallire prima del deploy incompatibile.

Non e compito di questo contratto dimostrare che 1990 sia il prezzo business corretto in ogni scenario. La formula di prezzo appartiene al comportamento interno del Pricing Service e richiede test di dominio e acceptance test del provider.

> **Collegamento con Hexagonal Architecture: la porta rende esplicita la dipendenza; il contract test verifica l adapter e il protocollo che attraversano quel boundary.**

5. Property-Based Testing: testare proprietà, non casi scelti

5.1 Dal test per esempio al test per proprieta

Un test example-based sceglie input specifici e un output specifico. E eccellente per comunicare scenari business: “dato un ordine da 100 EUR con sconto 10%, il totale e 90 EUR”. Ma non scala da solo allo spazio delle combinazioni.

Un property-based test descrive invece una relazione che deve rimanere vera per un insieme di input. Le librerie generano numerosi valori, eseguono la stessa proprieta e, quando trovano un fallimento, tentano di ridurlo a un controesempio piccolo. Il paper originale QuickCheck del 2000 ha reso popolare questo modello: proprieta come funzioni testate automaticamente su input generati.

5.2 I quattro elementi

- **Property:** l oracolo. E la relazione che deve essere vera.
- **Generator / strategy / arbitrary:** descrive come costruire input validi o interessanti.
- **Runner:** esegue molte prove e gestisce seed, replay, distribuzioni e limiti.
- **Shrinker:** minimizza il caso che fallisce.

fast-check descrive esplicitamente tre capacita centrali: generazione di input, esecuzioni multiple e shrinking. Hypothesis mantiene il concetto di minimal failing example: al termine dello shrinking riporta un caso ridotto che riproduce ancora la failure.

5.3 Shrinking

Lo shrinking e un elemento fondamentale per la praticita. Supponiamo che una proprieta fallisca su una lista di trenta item con prezzi, quantita e metadati casuali. Un buon shrinker prova a semplificare il caso: meno item, numeri piu piccoli, strutture piu semplici. Potresti finire con un solo item da prezzo 1 e quantita 2 che riproduce il bug.

> **Importante: lo shrinking rende leggibile un controesempio. Non rende corretta una property sbagliata e non corregge un generator che rappresenta male il dominio.**

6. Pattern di proprieta

6.1 Invarianti

Una condizione deve essere sempre vera per tutti gli input validi. Esempio: il totale monetario di un ordine valido non puo essere negativo.

6.2 Round-trip

Una trasformazione e la sua inversa dovrebbero preservare l informazione prevista. Esempio: deserialize(serialize(x)) == x, entro le normalizzazioni dichiarate.

6.3 Idempotenza

Applicare due volte un operazione produce lo stesso risultato di applicarla una volta: f(f(x)) == f(x). Utile per normalizzazione, canonicalizzazione, retry-safe operations e set semantics.

6.4 Metamorphic properties

Non conosci l output esatto, ma conosci una relazione tra input correlati. Se raddoppi tutte le quantita e non ci sono sconti non lineari, il totale atteso puo raddoppiare. Le metamorphic properties sono preziose quando calcolare un expected completo e costoso.

6.5 Monotonicita

Un cambiamento dell input non deve muovere l output nella direzione proibita. Per esempio, applicare uno sconto percentuale valido non deve aumentare il totale rispetto allo stesso ordine senza sconto.

6.6 Equivalenza / differential testing

Due implementazioni o due percorsi che dovrebbero essere equivalenti vengono confrontati. Utile durante refactoring, migrazioni e sostituzioni di algoritmi.

7. Esempi equivalenti in Python, TypeScript e PHP

Python + Hypothesis

```
from hypothesis import given, strategies as st

@given(st.lists(st.integers()))
def test_reverse_twice(xs):
assert list(reversed(list(reversed(xs)))) == xs
```

La property non elenca liste specifiche. Hypothesis genera molte liste, compresi casi vuoti, liste corte, valori piccoli e grandi, e prova a ridurre qualsiasi fallimento.

TypeScript + fast-check

```
import fc from "fast-check";

fc.assert(
fc.property(fc.array(fc.integer()), (xs) => {
const ys = [...xs].reverse().reverse();
expect(ys).toEqual(xs);
})
);
```

PHP + Eris

```
$this->forAll(
Generators::seq(Generators::int())
)->then(function ($xs) {
$ys = array_reverse(array_reverse($xs));
$this->assertSame($xs, $ys);
});
```

Eris e un port di QuickCheck per PHP/PHPUnit. Per progetti PHP moderni conviene comunque verificare attivamente compatibilita, manutenzione e integrazione con la propria versione di PHP/PHPUnit prima di adottarlo come dipendenza strategica.

8. Esempi + Properties + Contracts: la composizione corretta

La lezione 5 ci ha dato Rule + Examples. Questa lezione aggiunge un terzo strato: Properties. Le tre forme non hanno la stessa funzione.

- **Rule:** formula la politica o l invariante autorizzato dal dominio.
- **Examples:** rendono concreto il significato, coprono casi business importanti, migliorano comprensione e regression testing.
- **Properties:** estendono la ricerca a intere classi di input e combinazioni inattese.

- **Contracts:** proteggono il significato osservabile che attraversa un boundary tra componenti.

Quindi non sostituire indiscriminatamente test example-based con PBT. Un acceptance example puo essere piu leggibile da product owner e analisti; una property puo essere migliore per esplorare input. E non usare un contract test per provare la correttezza interna del provider.

8.1 Schema generativo: Schemathesis

Schemathesis mostra un punto di incontro interessante: legge OpenAPI o GraphQL e genera property-based API tests. Le sue versioni correnti usano Hypothesis per generare casi da schema, includendo fasi di examples, coverage, fuzzing e stateful testing; quando trova una failure, puo ridurla a un caso minimo riproducibile. Questo e utile per conformance e robustezza di API, ma non sostituisce automaticamente un consumer-driven contract: lo schema rimane la fonte da cui vengono generati i casi.

9. Failure mode e anti-pattern

9.1 Contract troppo specifico

Sintomo: aggiungere un campo opzionale o cambiare un dato che il consumer non usa rompe decine di contract test. Correzione: contratta cio da cui il consumer dipende realmente; usa matcher appropriati invece di snapshot rigidi dell intero payload.

9.2 Contract test che duplica test funzionali

Sintomo: il consumer codifica tutte le regole business del provider. Correzione: sposta la correttezza business nei test del provider. Il contract deve dimostrare comprensione condivisa del protocollo e delle risposte rilevanti.

9.3 Mocking delle due parti con la stessa supposizione

Se consumer e provider vengono testati entrambi contro mock scritti a mano sulla stessa interpretazione, possono condividere lo stesso errore. La provider verification deve incontrare l implementazione reale del provider.

9.4 Property debole

“Non deve lanciare eccezioni” e spesso un oracolo insufficiente. Molte implementazioni semanticamente sbagliate non crashano. Cerca invarianti, round-trip, relazioni metamorphic, monotonicita o confronto con un modello indipendente.

9.5 Generator che non rappresenta il dominio

Generare qualsiasi stringa quando il dominio ammette solo codici ISO, oppure qualsiasi combinazione di stato impossibile, produce rumore. I generatori devono essere parte del modello di test e meritano design.

9.6 Randomness non riproducibile

Un framework di PBT serio deve consentire replay e minimizzazione. Non confondere il PBT con uno script che chiama random() e fallisce in modo intermittente.

9.7 Test e implementazione con la stessa fonte di verita

Se l agent genera implementazione e test copiando la stessa formula, un errore di interpretazione puo essere duplicato. Mantieni l oracolo indipendente quando possibile: requirement, example autorizzato,

modello alternativo, contract esterno o property derivata da un invariante del dominio.

10. Quando usare, quando non usare, alternative

Contract Testing - quando usarlo

- Servizi o componenti deployati indipendentemente.
- Frontend e backend che evolvono con pipeline diverse.
- Eventi o messaggi consumati da team differenti.
- Molti consumer con dipendenze diverse sullo stesso provider.
- Necessita di bloccare breaking changes prima di integration environment o produzione.

Contract Testing - quando non basta o e eccessivo

- Monolite interno con refactoring atomico e nessun boundary di versione: spesso unit/integration test sono sufficienti.
- Flussi critici multi-sistema: serve comunque almeno una quota di E2E.
- Semantica business complessa del provider: il contract non sostituisce acceptance/unit test.

Property-Based Testing - quando usarlo

- Algoritmi, parser, serializer, normalizzatori, validatori, conversioni, calcoli monetari.
- Regole con invarianti semplici ma moltissime combinazioni.
- Refactoring in cui puoi confrontare vecchia e nuova implementazione.
- API schema-driven o protocolli con ampio spazio di input.

Property-Based Testing - quando non e la prima scelta

- Pochi casi discreti e gia esaustivi.
- Requisito espresso principalmente da esempi business che devono restare leggibili agli stakeholder.
- Operazioni lente o con side effect costosi, se non hai un seam che consenta una versione piu economica del test.

Alternative/complementi: example-based tests, mutation testing, fuzzing, model-based testing, schema conformance, integration test, E2E, static type checking e formal methods. La scelta dipende dalla domanda e dal rischio.

11. Il ruolo dell AI nel workflow

Ricordiamo la spina dorsale del corso: **Domain -> Requirements -> Spec -> Test -> Agent -> Verify**. Contract testing e property-based testing rafforzano la fase Test e la fase Verify, ma dipendono da decisioni precedenti corrette.

11.1 Contract Testing nel workflow di Matt Pocock

grill-with-docs / domain-modeling: chiarisci chi sono consumer e provider, quali dipendenze sono reali e quali UNKNOWN. **CONTEXT.md / ADR:** registra policy di compatibility e ownership. **to-spec:** rende espliciti boundary, contract e non-goals. **to-tickets:** separa consumer test, provider verification, broker/CI e migrazione. **implement/TDD:** implementa adapter e test. **code-review:** verifica che una

modifica di contract sia autorizzata e compatibile.

11.2 Property-Based Testing nel workflow

Le properties dovrebbero emergere da invarianti e relazioni gia autorizzate. L AI puo proporre property candidate, generatori, edge case e metamorphic relations. Ma una property inventata dall agent non diventa automaticamente una regola del dominio. Se l origine e incerta, rimane ASSUMED o UNKNOWN finche una fonte autorevole non la valida.

> **Regola per agenti: l agent puo ampliare la ricerca degli input; non deve ampliare silenziosamente l autorita della specifica.**

12. Laboratorio pratico - Pricing Service

Costruisci un mini-sistema con un core di pricing, una porta QuoteProvider e un adapter HTTP. Puoi farlo in PHP, Python o TypeScript.

- **Passo A:** definisci QuoteProvider.quote(items) -> Quote(total, currency).
- **Passo B:** scrivi tre example-based test: ordine vuoto; due quantita dello stesso item; sconto valido.
- **Passo C:** aggiungi almeno tre properties: total >= 0; permutare l ordine degli item non cambia il totale; aggiungere un item con qty=0 non cambia il totale.
- **Passo D:** definisci il contract HTTP minimo di POST /quotes usato dal consumer.
- **Passo E:** modifica il provider rinominando total in amount senza aggiornare il consumer. Osserva quale test deve fallire.
- **Passo F:** introduci un bug nel calcolo, per esempio uno sconto che puo rendere il totale negativo. Osserva se una property lo trova e quale controesempio viene ridotto.
- **Passo G:** classifica ogni failure: regola business, property, contract, integration o E2E.

Criterio di completamento

Il laboratorio e completato quando sai spiegare, senza parlare dello strumento, **perche** ogni test esiste, quale rischio riduce, qual e la sua fonte di verita e quale boundary o invariante protegge.

13. Takeaway

> **1. Contract Testing protegge la compatibilita osservabile tra componenti. Non dimostra tutta la correttezza del sistema.**
>
> **2. Property-Based Testing esplora classi di input attraverso properties, generatori e shrinking. Non sostituisce il giudizio su quali proprieta rappresentino davvero il dominio.**
>
> **3. Examples + Properties + Contracts sono complementari: significato, esplorazione e compatibilita.**
>
> **4. Hexagonal Architecture crea i seam; queste tecniche sfruttano quei seam per ottenere verifiche piu precise e meno costose.**

5. Con agenti AI, l indipendenza dell oracolo e ancora piu importante: non far generare all implementatore anche la verita senza una fonte autorizzata.

14. Fonti autorevoli e letture

Pact Docs - Introduction / Consumer-Driven Contracts
https://docs.pact.io/

Pact Docs - Writing Consumer Tests
https://docs.pact.io/consumer

Pact Docs - Provider verification
https://docs.pact.io/getting_started/provider_verification

Martin Fowler - Consumer-Driven Contracts: A Service Evolution Pattern
https://martinfowler.com/articles/consumerDrivenContracts.html

Martin Fowler - The Practical Test Pyramid
https://martinfowler.com/articles/practical-test-pyramid.html

Claessen & Hughes - QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs
https://research.chalmers.se/en/publication/237427

Hypothesis documentation - Glossary / minimal failing test case
https://hypothesis.readthedocs.io/en/latest/glossary.html

fast-check - What is Property-Based Testing?
https://fast-check.dev/docs/introduction/what-is-property-based-testing/

OpenAPI Specification 3.2.1
https://spec.openapis.org/oas/v3.2.1.html

Schemathesis - property-based API testing from OpenAPI/GraphQL
https://schemathesis.readthedocs.io/en/stable/

Eris - Property-based testing for PHP/PHPUnit
https://github.com/giorgiosironi/eris